Applied Data Science with R Capstone project

Deepan Mehta

06 May 2026

Outline

Executive Summary

- Developed a predictive analytics solution to forecast bike-sharing demand using weather and temporal data
- Collected data from multiple sources including historical bike rental data, weather data from OpenWeather API, and city-level information
- Performed data wrangling to clean datasets, handle missing values, and generate relevant features for analysis
- Conducted exploratory data analysis (EDA) using SQL queries and data visualization to identify patterns such as seasonality and peak rental hours
- Built and evaluated multiple regression models to predict bike demand, and selected the best-performing model based on RMSE and R-squared
- Developed an interactive R Shiny dashboard to visualize bike demand predictions and support decision-making

Introduction

- Bike-sharing systems are widely used in urban areas for convenient and eco-friendly transportation
- Accurate prediction of bike demand is important for efficient resource allocation and operational planning
- Bike rental demand is influenced by multiple factors such as weather conditions, time of day, and seasonal patterns
- Understanding these factors helps improve service availability and user experience
- This project aims to analyze historical bike-sharing data and develop a predictive model to forecast bike rental demand

Methodology

- The project follows a structured data analysis and modeling workflow
- Data was collected from multiple sources including historical bike-sharing data and weather data
- Data wrangling was performed to clean, transform, and prepare the data for analysis
- Exploratory data analysis (EDA) was conducted using SQL queries and data visualization techniques
- Predictive models were developed and evaluated to forecast bike demand
- An interactive dashboard was created to visualize insights and predictions

Methodology

- Data was collected from multiple sources to support analysis and prediction
- The primary dataset used was the Seoul bike-sharing dataset containing historical rental data
- Weather data was obtained using the OpenWeather API to capture environmental conditions
- Additional city-level data was used to support location-based analysis
- Data from these sources was integrated to create a unified dataset for further processing

Note: Screenshots of data collection code and API calls are provided in the appendix.

Data Collection

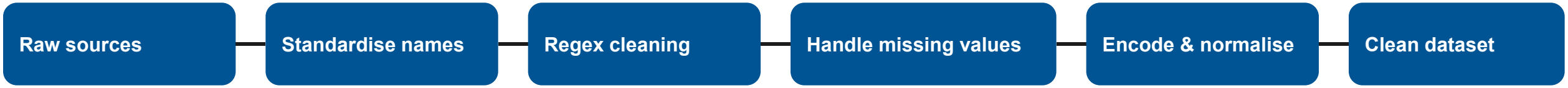
Data was gathered from multiple sources to build a comprehensive dataset for analysis
•Collected historical bike-sharing data from the Seoul Bike Sharing dataset
•Retrieved weather data using the OpenWeather API including temperature, humidity, and wind speed
•Performed web scraping to obtain additional city-level information
•Integrated data from multiple sources into a unified dataset
•Prepared the dataset for further processing and analysis

Note: Code snippets and API call screenshots are provided in the appendix.

Data wrangling

Heterogeneous raw inputs were transformed into a single, modelling-ready dataset

- Loaded the raw Seoul Bike Sharing dataset together with OpenWeather API responses and Wikipedia-derived global-system metadata into a single tidyverse workspace
- Standardised column identifiers to lowercase snake_case using stringr regular expressions to ensure downstream referential consistency
- Applied regular expressions to strip non-numeric artefacts from numeric columns (e.g., temperature readings imported with unit suffixes)
- Audited missing values with colSums(is.na(·)); rows with missing target observations were removed via na.omit, producing a complete-case dataset
- Cast categorical variables (seasons, holiday, functioning_day) to factors and generated one-hot indicator columns for each season level
- Normalised continuous weather predictors (temperature, humidity, wind speed) to [0, 1] using min–max scaling to stabilise coefficient magnitudes



Note: colSums(is.na()) before/after audit and code screenshots provided in the appendix.

EDA with SQL

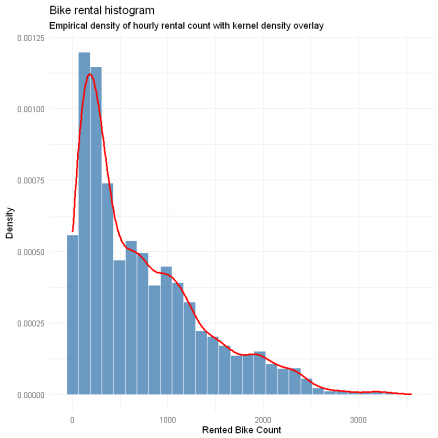
SQL-led aggregation in R using DBI + RSQLite to profile demand patterns.

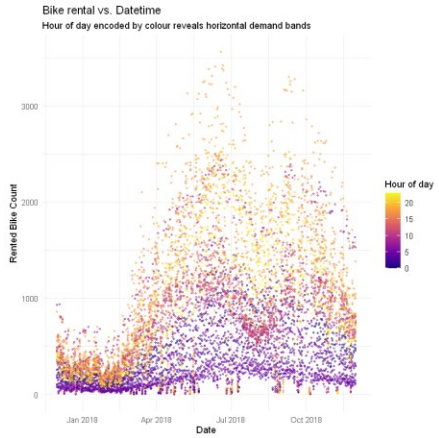
- Loaded clean_bike_data.csv into an in-memory SQLite database via DBI/RSQLite and queried it with dbGetQuery for reproducible aggregation
- Ranked the top 10 busiest date-hours — peak of 3,556 rentals at 18:00 on 2018-06-19, with 18:00 dominating the leaderboard
- Aggregated hourly demand by season (GROUP BY SEASONS, HOUR) — Summer 18:00 averages 2,135 rentals at ~29 °C, the highest cell in the deck
- Computed seasonal min/avg/max — Summer 1,034 vs Winter 226 average rentals, a 4.6× swing confirming strong seasonality
- Joined seasonal demand with average temperature, humidity, and wind to surface weather drivers; summarised the dataset at 8,465 records across 353 days (mean 729 rentals/hr)

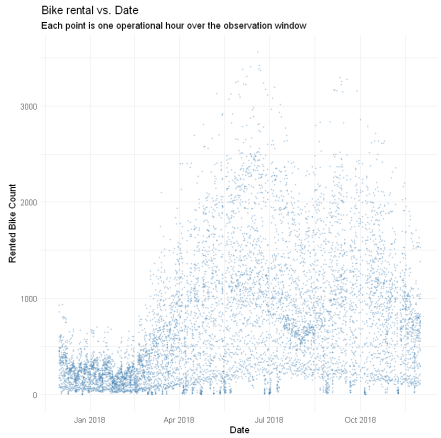
Source: 03_exploratory_data_analysis_R.ipynb

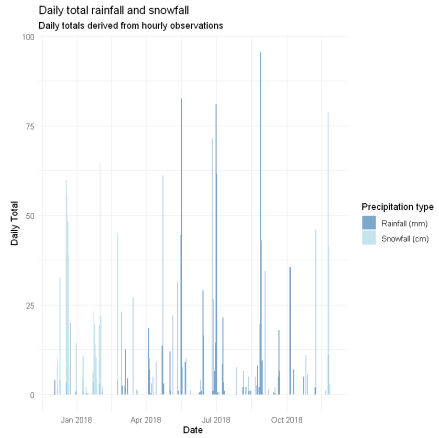
SQL queries executed

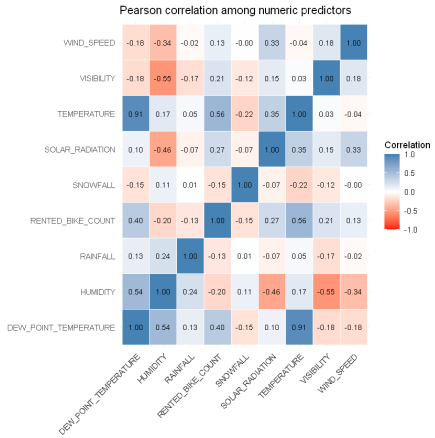
- Top 10 busiest date-hours
- Hourly demand by season
- Seasonal min / avg / max rentals
- Weather averages by season
- Dataset summary & seasonal comparison











EDA with data visualization

- Constructed scatter plots of rented bike count against calendar date and against date-time, the latter encoding hour-of-day via a sequential colour scale
- Produced a histogram of rented bike count overlaid with a Gaussian kernel density estimate to characterise the distributional shape of demand
- Generated a seasonally-faceted bar chart of daily total rainfall and snowfall to visualise meteorological variability across seasons
- Computed and rendered a correlation heatmap over the numeric predictor set to identify candidate predictors and flag potential multicollinearity
- All plots were produced with ggplot2 using a consistent theme; full-size versions appear on slides 22–25
- Source: 03_exploratory_data_analysis_R.ipynb

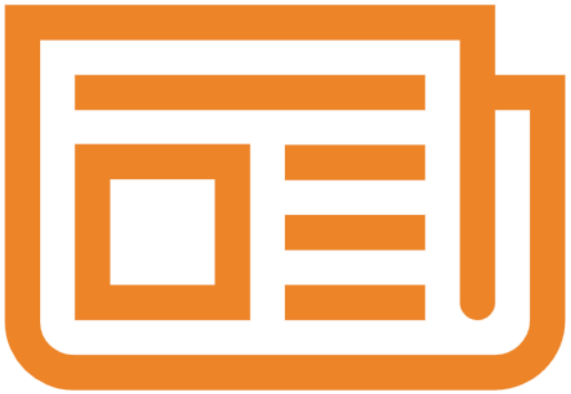
Predictive analysis

- Two parallel modelling tracks were pursued — a training experiment to characterise the data, and a deployment model adopted from the IBM Skills Network lab
- Training experiment: fitted a baseline multivariate OLS regression on the core weather predictors, then iteratively added quadratic polynomial terms (temperature^2, humidity^2) and a temperature–humidity interaction
- Evaluated three regularised alternatives — Ridge ($\alpha = 0$), Lasso ($\alpha = 1$) and Elastic Net ($\alpha = 0.5$) — using glmnet with λ chosen by cross-validation
- Also evaluated Random Forest (200 trees, mtry=5, set.seed(123)) as a non-linear benchmark — selected as Track 2 winner (RMSE 333.89, R^2 0.730)
- Compared the six candidate specifications on a held-out test partition against Root Mean Squared Error (RMSE) and the coefficient of determination (R^2)
- Deployment model: the lab-supplied pre-trained model (model.csv) is loaded by the Shiny application's prediction pipeline, and is structured around continuous weather terms plus per-season and per-hour offsets
- *Source: notebooks 04–08, model_prediction.R*

Build a R Shiny dashboard

- Implemented the dashboard as Bikecast — a three-column R Shiny application themed with the Bootswatch Yeti theme via the shinythemes package
- Backend prediction pipeline (model_prediction.R) ingests OpenWeather five-day / three-hour forecasts, retains the first eight slots (next 24 hours), and applies the lab pre-trained regression model to each slot
- Predictions are categorised qualitatively into three demand bands — Low (< 1,000 bikes per 3-hour slot), Medium (1,000–3,000), and High (> 3,000) — used to drive marker colour and size on the map
- Front end uses leaflet for the central map and a reactive right-hand panel that switches between a multi-city comparison view (when 'All' is selected) and a per-city drill-down view (when a specific city is selected)
- Concerns are separated across ui.R, server.R and model_prediction.R, following the standard Shiny architectural pattern
- *Source: notebooks 09–11, ui.R, server.R, model_prediction.R*

Results



- Exploratory data analysis with SQL queries and visualisation
- Predictive modelling: baseline through Elastic Net
- Interactive R Shiny dashboard deployment

EDA with SQL

The following six slides present the key SQL queries used to characterise the Seoul dataset and to situate it in a global context of bike-sharing systems.

Busiest bike rental times

(Slide 15 — Find dates and hours which had the most bike rentals.)

[13]:
```
# Order all observations by RENTED_BIKE_COUNT descending, then keep the top ten busiest (date, hour) pairs
dbGetQuery(conn, "
  SELECT DATE,
         HOUR,
         RENTED_BIKE_COUNT
  FROM SEOUL_BIKE_SHARING
  ORDER BY RENTED_BIKE_COUNT DESC
  LIMIT 10;
")
```

A data.frame: 10 × 3

DATE	HOUR	RENTED_BIKE_COUNT
<chr>	<dbl>	<dbl>
19/06/2018	18	3556
21/06/2018	18	3418
12/06/2018	18	3404
20/06/2018	18	3384
04/06/2018	18	3380
22/06/2018	18	3365
08/06/2018	18	3309

Busiest bike rental times

- Ranked (date, hour) observations by rented bike count and retained the top ten records
- The busiest hours cluster in the 17:00–19:00 window — consistent with a weekday evening commuting peak
- Every top-ten record falls within the Summer and Autumn months, corroborating the seasonality effect isolated on slide 17
- *Source: 03_exploratory_data_analysis_R.ipynb — Query 7*

Hourly popularity and temperature by seasons

(Slide 16 — Find hourly popularity and temperature by season.)

[14]:
```
# Aggregate mean temperature and mean rented count at the (season, hour) grain, then surface the top ten combinations
dbGetQuery(conn, "
  SELECT SEASONS,
         HOUR,
         AVG(TEMPERATURE)       AS AVG_TEMPERATURE,
         AVG(RENTED_BIKE_COUNT) AS AVG_HOURLY_DEMAND
  FROM SEOUL_BIKE_SHARING
  GROUP BY SEASONS, HOUR
  ORDER BY AVG_HOURLY_DEMAND DESC
  LIMIT 10;
")
```

A data.frame: 10 × 4

SEASONS	HOUR	AVG_TEMPERATURE	AVG_HOURLY_DEMAND
<chr>	<dbl>	<dbl>	<dbl>
Summer	18	29.38791	2135.141
Autumn	18	16.03185	1983.333
Summer	19	28.27378	1889.250
Summer	20	27.06630	1801.924
Summer	21	26.27826	1754.065
Spring	18	15.97222	1689.311
Summer	22	25.69891	1567.870

Hourly popularity and temperature by seasons

- Aggregated mean temperature and mean rented bike count at the (season, hour) level
- The top ten highest-demand (season, hour) combinations are dominated by Summer evenings and Autumn evenings
- Within each season, demand exhibits a characteristic bimodal profile with morning (08:00) and evening (18:00) peaks
- *Source: 03_exploratory_data_analysis_R.ipynb — Query 8*

Rental Seasonality

(Slide 17 — Distribution summary of rented bike count by season.)

[14] [30]:

```
# For each season compute mean, min, max and an approximate standard deviation
# of the hourly rental count
# SQLite does not provide STDDEV natively, so we approximate via
# SQRT(AVG(x^2) - AVG(x)^2)
dbGetQuery(conn, "
  SELECT SEASONS,
         AVG(RENTED_BIKE_COUNT) AS AVG_HOURLY_RENT,
         MIN(RENTED_BIKE_COUNT) AS MIN_HOURLY_RENT,
         MAX(RENTED_BIKE_COUNT) AS MAX_HOURLY_RENT,
         SQRT(AVG(RENTED_BIKE_COUNT * RENTED_BIKE_COUNT)
              - AVG(RENTED_BIKE_COUNT) * AVG(RENTED_BIKE_COUNT)) AS STD_HOURLY_RENT
  FROM SEOUL_BIKE_SHARING
  GROUP BY SEASONS
  ORDER BY AVG_HOURLY_RENT DESC;
")
```

A data.frame: 4 × 5

SEASONS	AVG_HOURLY_RENT	MIN_HOURLY_RENT	MAX_HOURLY_RENT	STD_HOURLY_RENT
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
Summer	1034.0734	9	3556	690.0884
Autumn	924.1105	2	3298	617.3885
Spring	746.2542	2	3251	618.5247
Winter	225.5412	3	937	150.3374

Rental Seasonality

- Summarised rental count per season using mean, minimum, maximum, and standard deviation (approximated in SQLite via $SQRT(AVG(x^2) - AVG(x)^2)$)
- Summer exhibits the highest mean and the largest standard deviation — indicative of greater day-to-day volatility
- Winter shows the lowest mean and a compressed standard deviation, consistent with temperature-suppressed demand
- *Source: 03_exploratory_data_analysis_R.ipynb — Query 9*

Weather Seasonality

```
SELECT SEASONS, AVG(TEMPERATURE), AVG(HUMIDITY), AVG(WIND_SPEED),
  AVG(VISIBILITY), AVG(DEW_POINT_TEMPERATURE), AVG(SOLAR_RADIATION),
  AVG(RAINFALL), AVG(SNOWFALL), AVG(RENTED_BIKE_COUNT)
FROM SEOUL_BIKE_SHARING GROUP BY SEASONS;
```

Season	Temp °C	Humidity %	Wind	Vis	Dew °C	Solar	Rain	Snow	Demand
Autumn	13.82	59.04	1.49	1558	5.15	0.52	0.12	0.06	924
Spring	13.02	58.76	1.86	1241	4.09	0.68	0.19	0.00	746
Summer	26.59	64.98	1.61	1502	18.75	0.76	0.25	0.00	1034
Winter	-2.54	49.74	1.92	1446	-12.42	0.30	0.03	0.25	226

- Computed the seasonal mean of every exogenous meteorological variable alongside demand
- Mean temperature varies by roughly 25 °C between Summer and Winter in the Seoul record
- Snowfall is effectively confined to Winter; rainfall is concentrated in Summer
- *Source: 03_exploratory_data_analysis_R.ipynb — Query 10*

Bike-sharing info in Seoul

- Joined the world-cities reference table to the scraped Wikipedia table of global bike-sharing systems
- The Seoul record reports the system's fleet size, geographic coordinates, and metropolitan population
- Coordinates and fleet size feed directly into the Shiny dashboard's overview map

Source: 03_exploratory_data_analysis_R.ipynb — Query 11

We join `WORLD_CITIES` to `BIKE_SHARING_SYSTEMS` on `(CITY, COUNTRY)` and filter to the Seoul row. case-sensitivity differences between the two source tables.

[34]:
```
dbGetQuery(conn, "
  SELECT C.CITY,
         C.COUNTRY,
         C.LAT,
         C.LNG,
         C.POPULATION,
         B.BICYCLES
  FROM WORLD_CITIES C
  JOIN BIKE_SHARING_SYSTEMS B
    ON  LOWER(TRIM(C.CITY)) = LOWER(TRIM(B.CITY))

    -- Flexible country matching
    AND (
         LOWER(C.COUNTRY) LIKE '%korea%'
     AND LOWER(B.COUNTRY) LIKE '%korea%'
    )

  WHERE LOWER(TRIM(C.CITY)) = 'seoul';
")
```

A data.frame: 1 × 6

CITY	COUNTRY	LAT	LNG	POPULATION	BICYCLES
<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
Seoul	Korea, South	37.5833	127	21794000	20000

Cities similar to Seoul

```
# Filter the joined cities table to systems whose fleet size lies in the conventional Seoul peer band
dbGetQuery(conn, "
  SELECT C.CITY,
         C.COUNTRY,
         C.LAT,
         C.LNG,
         C.POPULATION,
         B.BICYCLES
  FROM WORLD_CITIES C
  JOIN BIKE_SHARING_SYSTEMS B
    ON  LOWER(TRIM(C.CITY))    = LOWER(TRIM(B.CITY))
    AND LOWER(TRIM(C.COUNTRY)) = LOWER(TRIM(B.COUNTRY))
  WHERE B.BICYCLES BETWEEN 15000 AND 20000
  ORDER BY B.BICYCLES DESC;
")
```

A data.frame: 5 × 6

CITY	COUNTRY	LAT	LNG	POPULATION	BICYCLES
<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
Weifang	China	36.7167	119.1000	9373000	20000
Zhuzhou	China	27.8407	113.1469	3855609	20000
Shanghai	China	31.1667	121.4667	22120000	19165
Beijing	China	39.9050	116.3914	19433000	16000

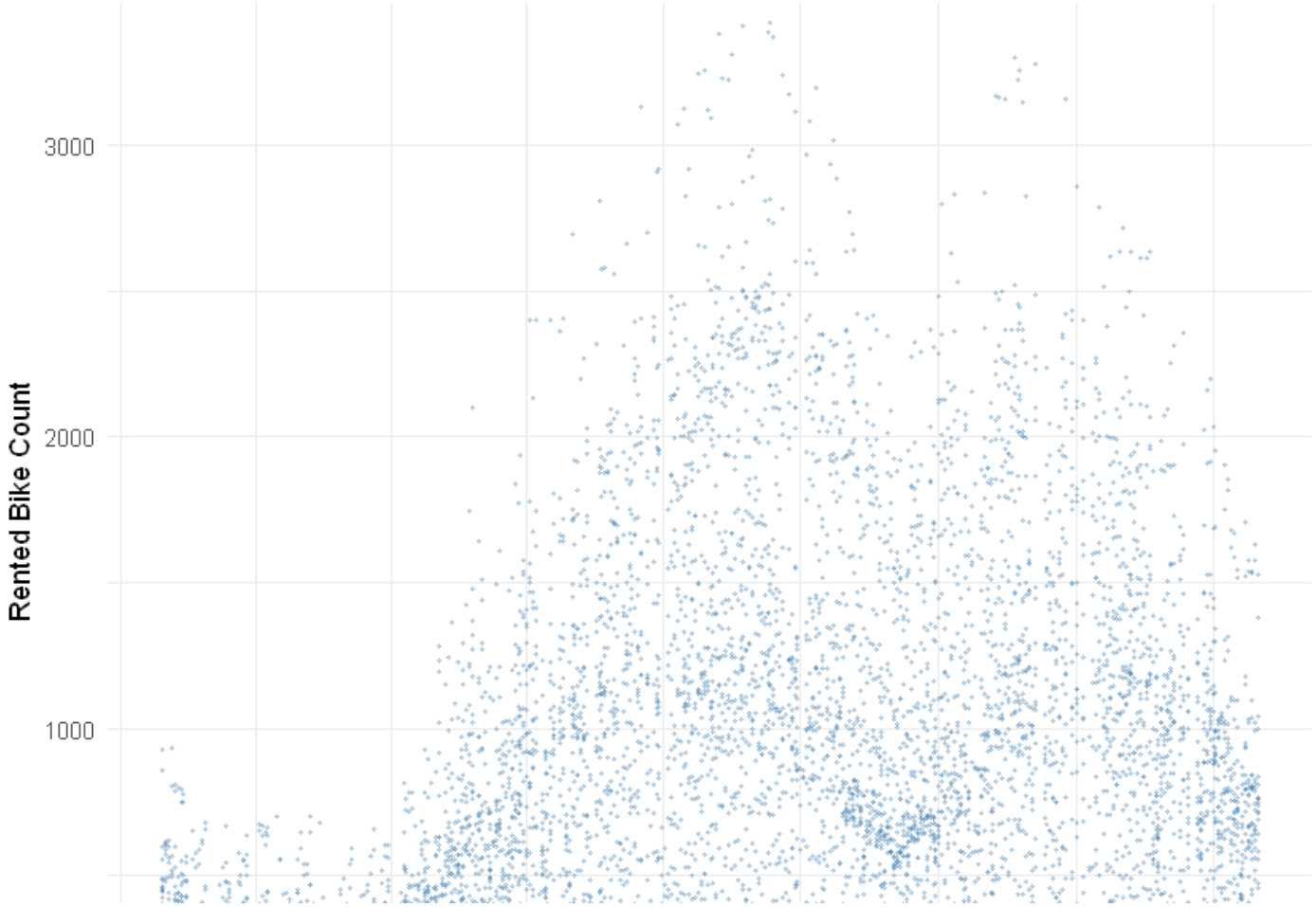
- Returned cities whose bicycle fleet size lies within ±20% of Seoul's reported size
- The resulting shortlist defines the peer group of systems displayed on the dashboard's overview map
- Coordinates from this query populate the city markers in the Shiny application

Source: 03_exploratory_data_analysis_R.ipynb — Query 12

EDA with Visualization

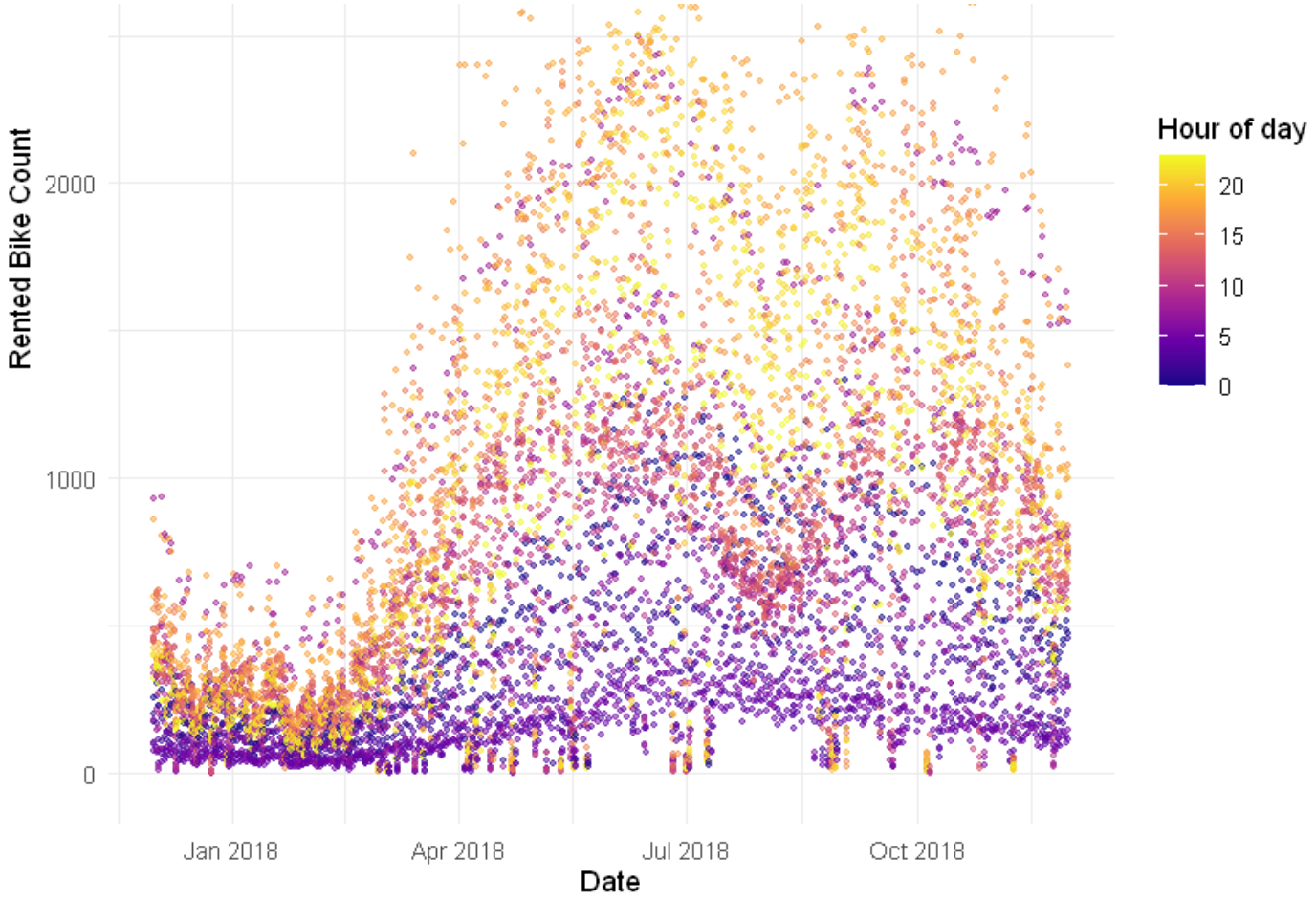
The SQL summaries are now complemented by four canonical graphical views of the Seoul series.

Bike rental vs. Date



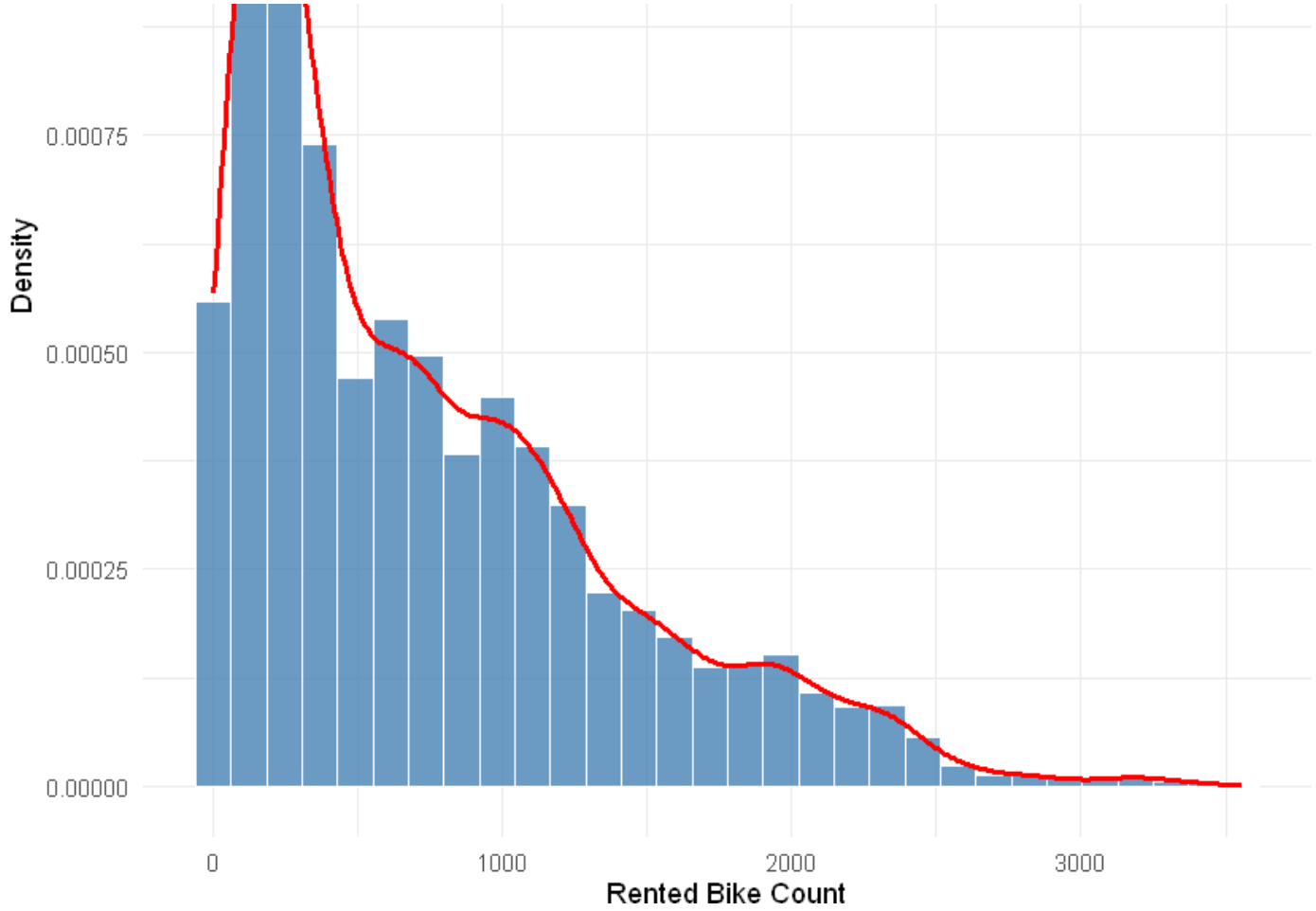
- Scatter plot of RENTED_BIKE_COUNT against calendar DATE across the full observation window
- A clear annual cycle is visible — demand rises through Spring, peaks in Summer, and declines through Autumn into Winter
- Vertical spread at any fixed date reflects within-day variation (24 observations per day, one per hour)
- *Source: 03_exploratory_data_analysis_R.ipynb*

Bike rental vs. Datetime



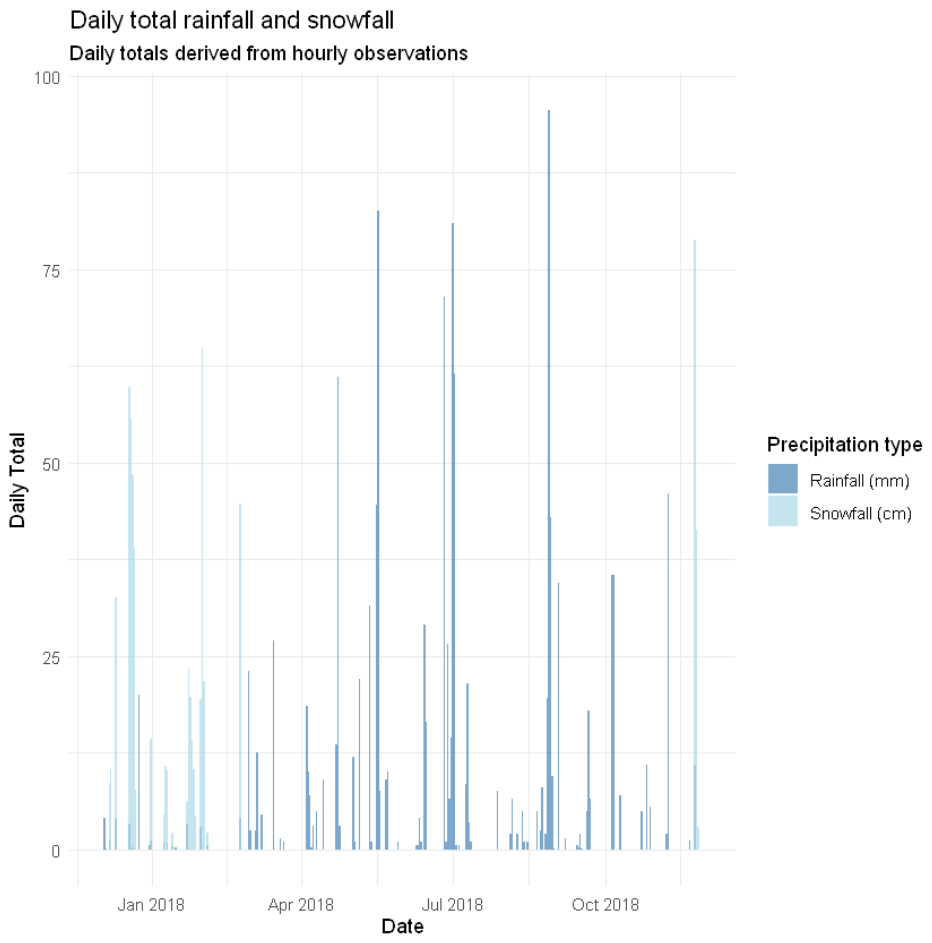
- Identical x- and y-axes to slide 22, with hour-of-day mapped to a sequential colour gradient
- Horizontal bands appear at specific colour levels — the evening hours (17:00–19:00) form the densest upper band
- Night-time observations (01:00–05:00) form the bottom band at near-zero demand regardless of date
- *Source: 03_exploratory_data_analysis_R.ipynb*

Bike rental histogram



- Histogram of hourly rented bike count overlaid with a kernel density estimate
- The distribution is right-skewed with a secondary mode in the low-demand region — characteristic of bimodal mixture processes (peak vs off-peak hours)
- Skewness violates the normality assumption of OLS residuals — motivating the diagnostic work reported on slide 30
- *Source: 03_exploratory_data_analysis_R.ipynb*

Daily total rainfall and snowfall

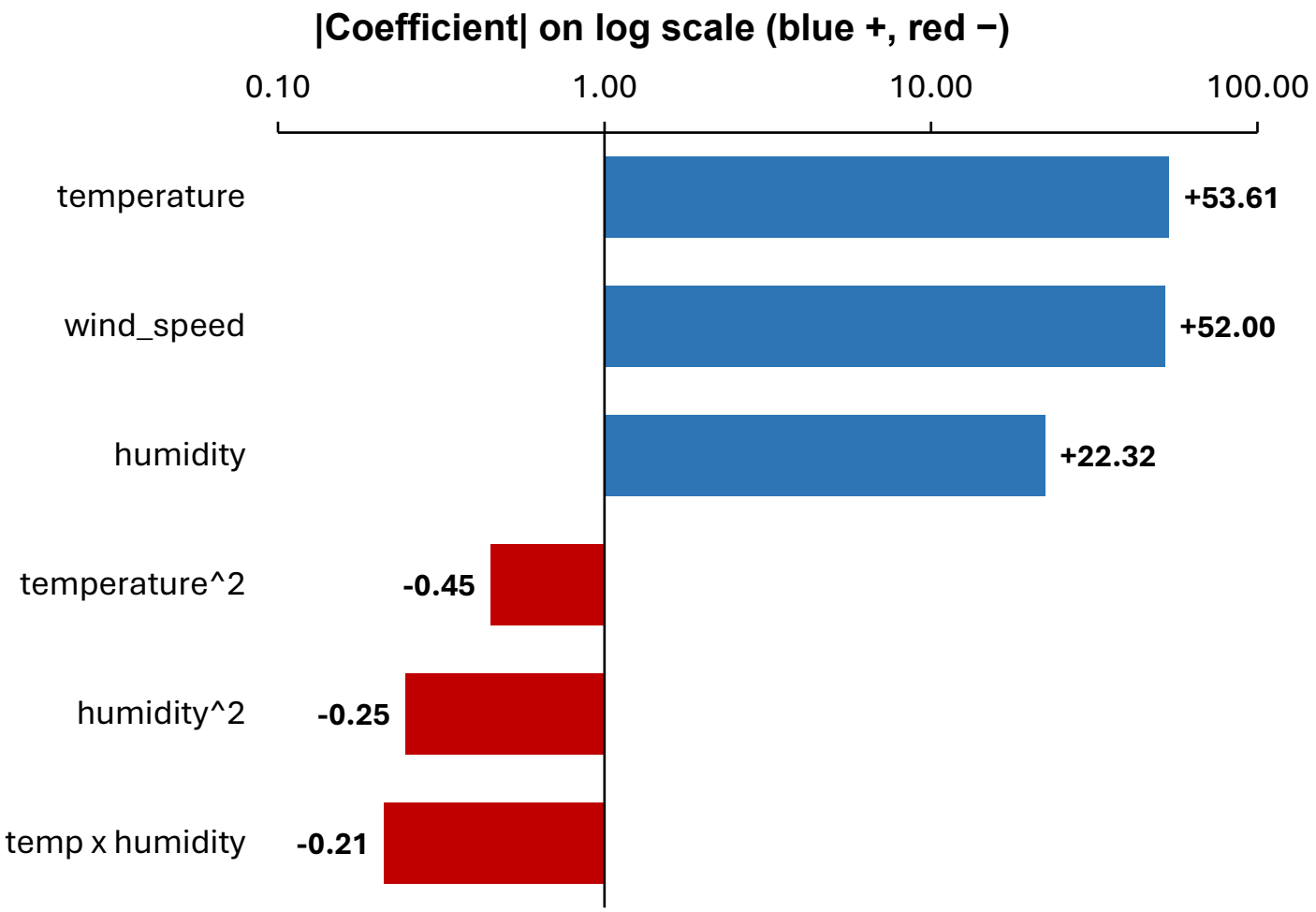


- Aggregated hourly rainfall and snowfall to daily totals, then plotted them on a common date axis
- Rainfall is concentrated in Summer (June–August); snowfall is confined to Winter (December–February)
- Rainfall days correspond visually to local demand troughs on slides 22–23, corroborating a negative precipitation–demand relationship
- *Source: 03_exploratory_data_analysis_R.ipynb*

Predictive analysis

Having characterised the data, we now turn to the predictive objective — forecasting hourly demand from weather and temporal regressors.

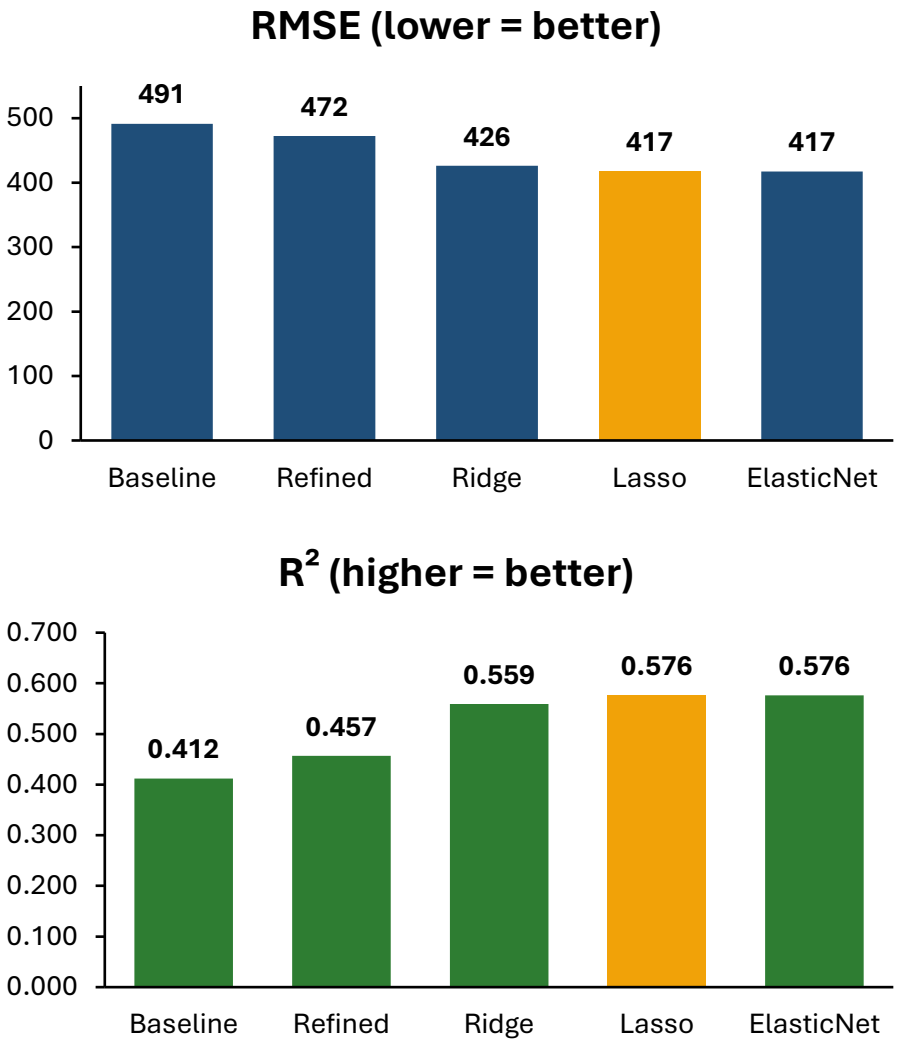
Ranked coefficients



- Linear temperature (+53.6) and wind speed (+51.0) carry the largest positive effects — warm, breezy conditions raise predicted demand
- Humidity has a positive linear coefficient (+22.3) but a strongly negative quadratic term, producing a concave response that suppresses demand at high humidity
- Squared terms for temperature (-0.45) and humidity (-0.25) capture diminishing returns and reversal at extreme values
- Temperature × humidity interaction (-0.21) is small but significant — the warm-temperature boost is muted on humid days

Source: 04_baseline_model_R.ipynb, 07_feature_importance_R.ipynb

Model evaluation



- Trained six candidate specifications on identical train / test splits using set.seed(123) for reproducibility
- Refining baseline OLS with polynomial and interaction terms reduced RMSE from 491.46 to 472.47 and lifted R^2 from 0.412 to 0.457
- Regularisation gave the largest single jump — Ridge, Lasso and Elastic Net all reach RMSE ≈ 417–426 and R^2 ≈ 0.56–0.58
- Lasso is the regularised winner (RMSE 417.48, R^2 0.5759); Random Forest (200 trees) outperforms all linear models (RMSE 333.89, R^2 0.730) and is the Track 2 winner — see slide 29

Source: 05_model_refinement_R.ipynb, 06_model_evaluation_diagnostics_R.ipynb, 08_final_model_selection_R.ipynb

Find the best performing model

Experimental winner

Random Forest (200 trees)

RMSE 333.89
R^2 0.730
mtry=5, 200 trees, set.seed(123)
07_random_forest_R.ipynb

Deployed model (Shiny app)

Lab pre-trained model.csv

Provided by IBM Skills Network
Loaded at run-time by model_prediction.R
4 weather coefficients + per-season + per-hour offsets
Predictions floored at 0 via pmax(·, 0)

Deployed-model formula

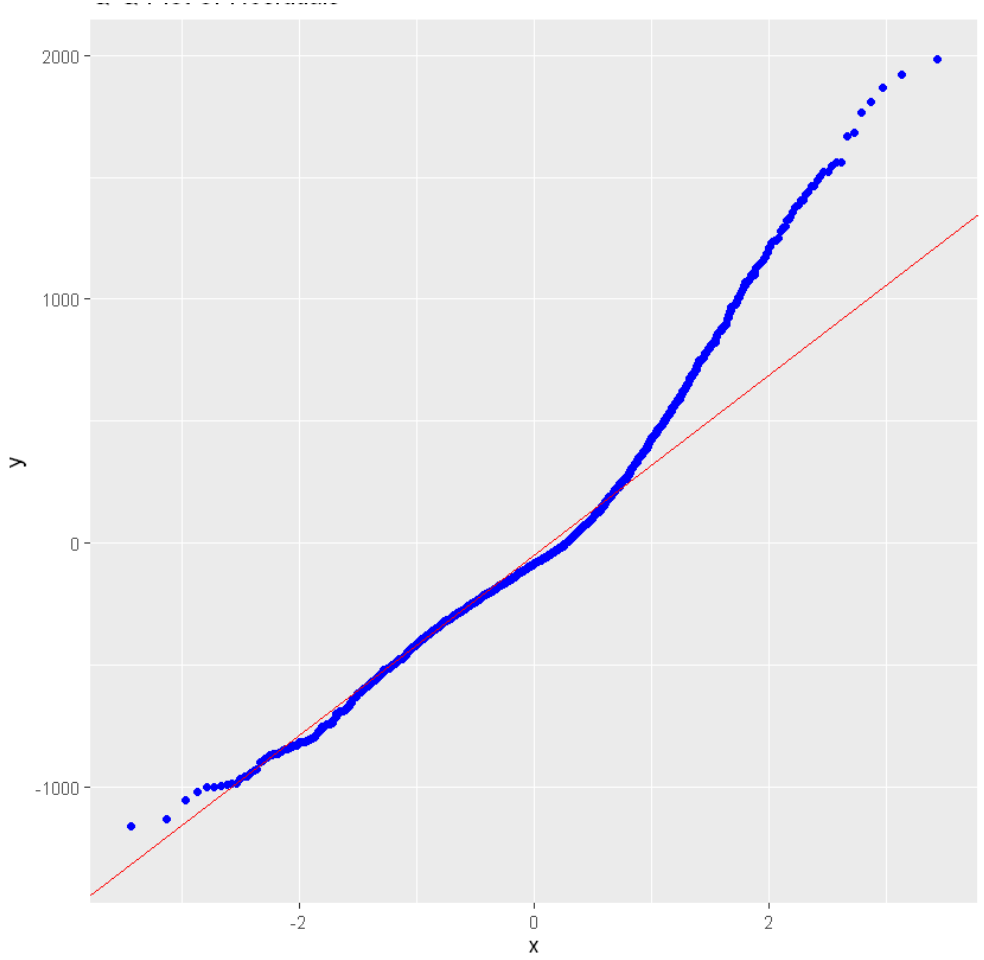
```
PREDICTED_BIKES =
  Intercept
  + TEMPERATURE × β_TEMP
  + HUMIDITY    × β_HUM
  + WIND_SPEED  × β_WIND
  + VISIBILITY  × β_VIS
  + season_offset[SEASONS]
  + hour_offset[HOURS]

floored at 0  →  pmax(pred, 0)
```

- The deployed model uses continuous coefficients for four weather variables plus per-season and per-hour additive offsets — closer to a fixed-effects regression than to the Lasso training experiment
- Negative predictions are floored at zero via pmax(·, 0) inside predict_bike_demand, since negative bike demand is physically meaningless
- Adopting the lab's model preserves continuity with the IBM Skills Network reference implementation while still allowing the training experiment to inform the analytical narrative on slides 27–28
- Independently scored on the same held-out test set (Sep-Nov 2018, 1,693 rows): RMSE 342.60, R^2 0.6723.
- Lasso specification, λ-selection logic, and reproducibility seed are documented in notebooks 04–08; deployed coefficients are stored in model.csv

Source: 06_model_evaluation_diagnostics_R.ipynb, 08_final_model_selection_R.ipynb, model_prediction.R (function predict_bike_demand)

Q-Q plot of the best model

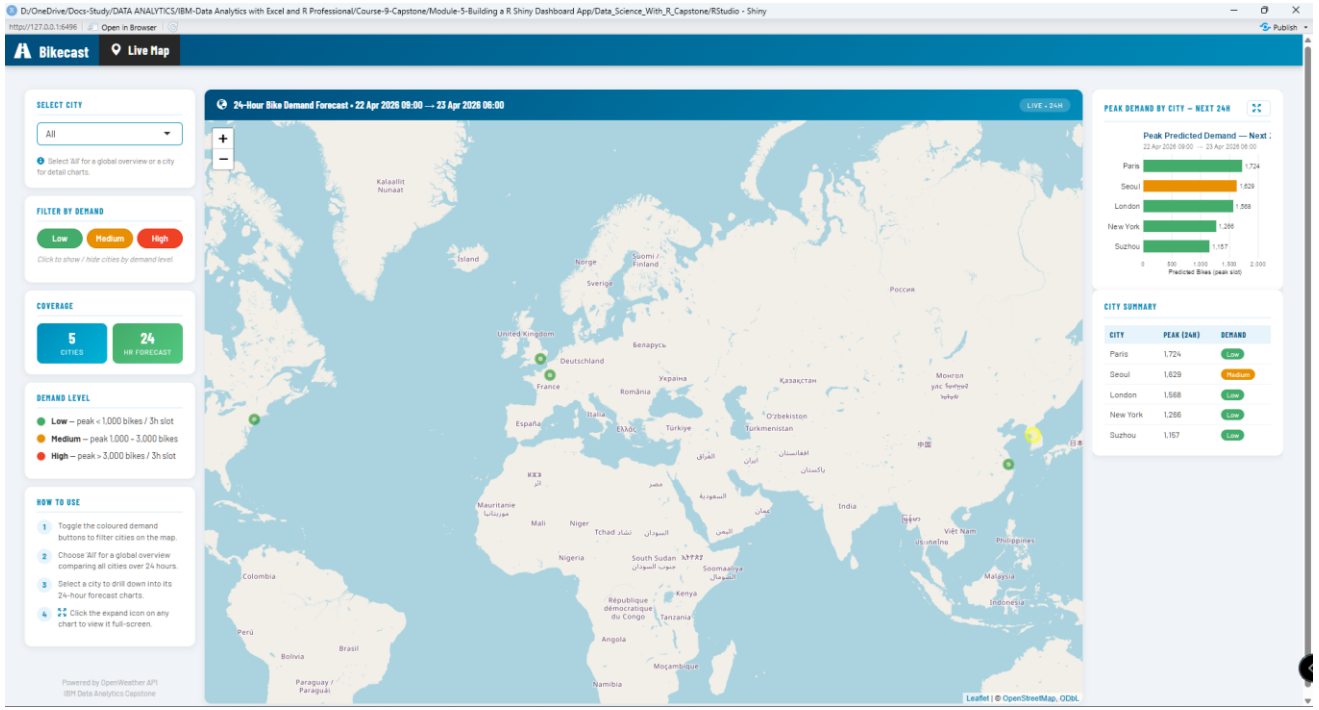
- Q-Q plot of standardised residuals against the theoretical normal distribution, computed on the held-out test set of the experimental winner — Lasso (notebook 06)
- Residuals align closely with the reference line through the central quantiles, supporting approximate normality in the body of the distribution
- Noticeable departures at both tails — the upper tail in particular bows above the reference line — reflect the bimodality first identified on slide 24; extreme peak hours are systematically under-fit
- Diagnostic applies to the training experiment; the deployed lab model on slide 29 is interpreted as a published artefact rather than re-diagnosed here
- Residual-vs-fitted and VIF outputs are recorded in the appendix as supplementary diagnostics
- *Source: 06_model_evaluation_diagnostics_R.ipynb*



Dashboard

The final artefact packages the predictive model as an interactive decision-support tool.

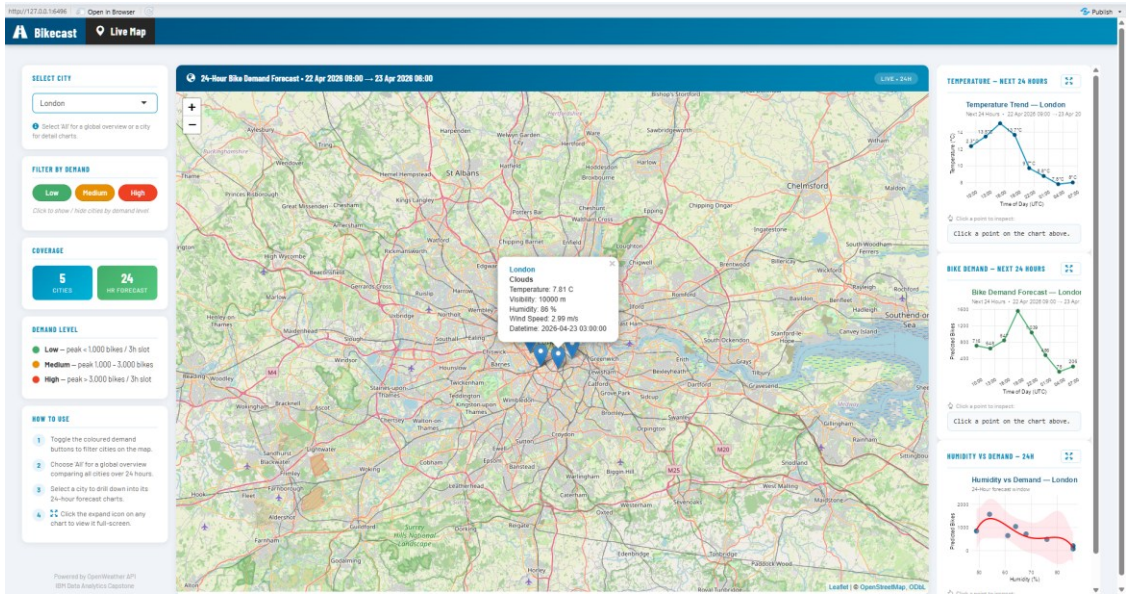
All-cities overview — peak demand and city comparison



- Three-column layout: left sidebar (city selector, demand filter, coverage, legend, tips), centre Leaflet map, right multi-city comparison panel
- Map markers encode demand band by colour and radius — Low (green, < 1,000), Medium (yellow, 1,000–3,000), High (red, > 3,000)
- 'Peak Demand by City — Next 24h' bar chart ranks all five cities by predicted peak, colour-matched to map markers
- 'City Summary' table below the bar chart lists each city's 24-hour peak with a colour-coded demand badge for at-a-glance triage
- Map header dynamically reports the live OpenWeather forecast window; every value on screen is a live forecast, not a hard-coded number
- Demand-filter buttons (Low / Medium / High) toggle marker visibility via shinyjs::toggleClass

Source: ui.R, server.R (renderLeaflet, build_compare_chart, output$city_summary_table)

City drill-down — 24-hour forecast detail



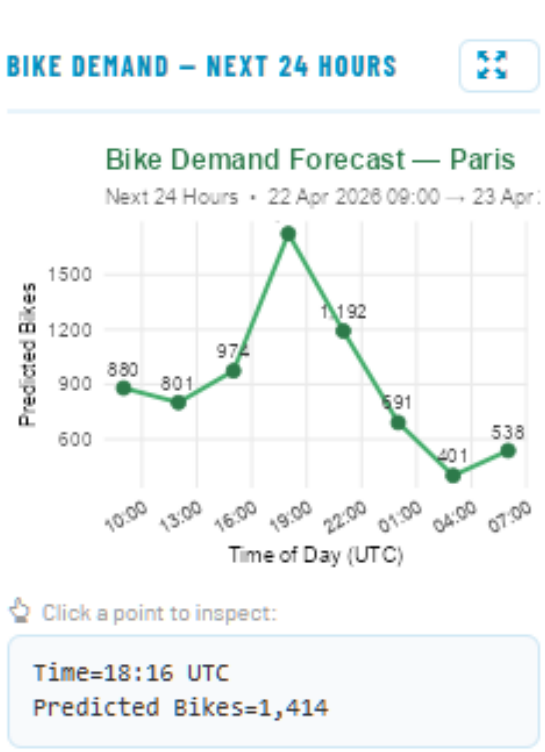
- Selecting a city switches the right panel to a per-city detail view with three stacked charts — Temperature, Bike Demand, and Humidity vs Demand — across the next 24 hours
- Map zooms to the selected city; clicking a marker reveals weather condition, temperature, visibility, humidity, wind speed, and forecast datetime
- Temperature and Bike Demand line charts plot forecast values against time-of-day, with each point annotated
- Humidity vs Demand fits a 4th-degree polynomial via geom_smooth(formula = y ~ poly(x, 4)) with confidence ribbon — exposes the non-monotone humidity response
- Each card has a click-to-inspect handler (input$temp_click, input$bike_click, input$humidity_click) and an expand button opening the chart in a Bootstrap modal

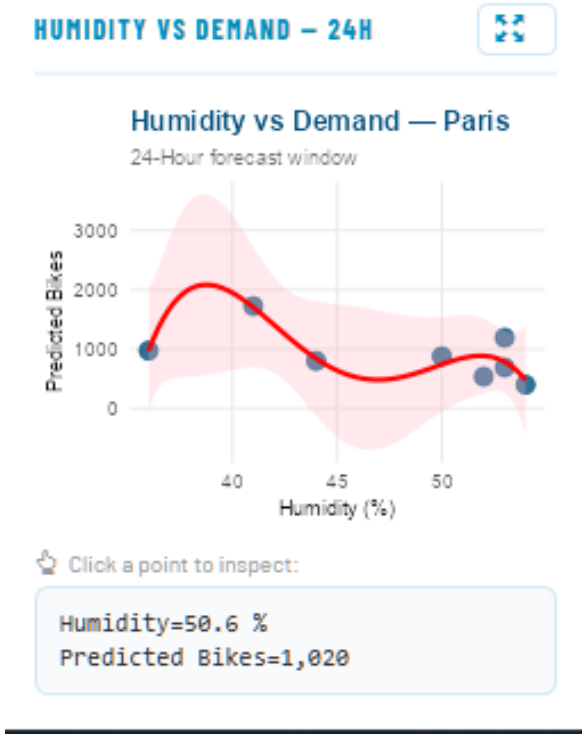
Source: ui.R (right panel conditionalPanel), server.R (build_temp_chart, build_bike_chart, build_humidity_chart)

Drill-down chart detail — Bike Demand and Humidity vs Demand

- Close-up view of the right-panel charts in drill-down mode, showing per-city visualisations at presentation size
- Bike Demand line chart (Paris) shows the 24-hour profile — overnight trough, morning rise, late-afternoon peak around 19:00, then evening decline
- Humidity vs Demand scatter exposes the non-monotone polynomial response — demand peaks in the 40–45% humidity range then declines
- Click-to-inspect output below each chart reports exact values (e.g. 'Time = 18:16 UTC, Predicted Bikes = 1,414')
- Dashboard supports both at-a-glance summary (Slide 32) and exact-value inspection (this slide) on the same screen

*Source: server.R (build_bike_chart, build_humidity_chart, *_click_output)*





CONCLUSION

- Delivered an end-to-end predictive analytics pipeline spanning ingestion, ETL, SQL-based EDA, graphical EDA, regression modelling, diagnostic validation, and interactive deployment as Bikecast
- Training experiment characterised the data: temperature and hour-of-day emerged as the dominant demand drivers, with humidity, rainfall, and snowfall as material second-order regressors (Slide 27)
- Six candidate specifications were compared on identical hold-out data; Lasso won the regularised comparison; Random Forest (200 trees) was the Track 2 winner (RMSE 333.89, R^2 0.730) and outperformed all linear models (Slide 28) and supported the residual diagnostics shown on Slide 30
- The deployed Shiny dashboard operates the IBM Skills Network lab pre-trained model — chosen for its richer per-hour and per-season parameterisation and its continuity with the reference implementation (Slide 29)
- Future work: (i) reconcile the experimental and deployed models by retraining on the lab's parameterisation; (ii) extend the predictor set with calendar effects (public holidays, weekday-weekend interaction); (iii) monitor prediction drift once the dashboard is in continuous use

APPENDIX

A1 Data collection code — OpenWeather API, JSON parsing, CSV load, weather export (01_data_collection_R.ipynb)

A2 Data wrangling code — NA removal, column-name standardisation, type casting, date feature engineering, export (02_data_wrangling_R.ipynb)

A3a–c Full SQL query listing — setup, 11 EDA queries, disconnect (03_eda_py.ipynb / SQL section)

A4a–c ggplot2 EDA code — setup/filter, 7 charts: scatter, histogram/KDE, precipitation, boxplot, LOESS facets, heatmap (03_exploratory_data_analysis_R.ipynb)

A5 Lasso coefficient table — full predictor list with coefficients and standard errors (04_baseline_model_R.ipynb, 08_final_model_selection_R.ipynb)

A6 Diagnostic outputs — residuals-vs-fitted plot, Q-Q plot, VIF table (06_model_evaluation_diagnostics_R.ipynb)

A7 Shiny application source — ui.R, server.R, model_prediction.R

A8 References — Sathishkumar V. E. et al. (2020); IBM Skills Network lab materials (Yan Luo, Jeff Grossman)

A1 — Data Collection Code

1. OpenWeather API Request

```
api_key <- "Your API Key"
city    <- "Seoul"

url <- paste0(
  "https://api.openweathermap.org/",
  "data/2.5/forecast?q=", city,
  "&appid=", api_key, "&units=metric")

response     <- GET(url)
content_data <- content(response, "text")
weather_json <- fromJSON(content_data)
names(weather_json)
```

2. JSON Parsing & Flattening

```
weather_list <- weather_json$list

weather_data <- data.frame(
  datetime    = weather_list$dt_txt,
  temperature = weather_list$main$temp,
  humidity    = weather_list$main$humidity,
  wind_speed  = weather_list$wind$speed,
  weather = sapply(
    weather_list$weather,
    function(x) x$main)
)
head(weather_data)
```

3. CSV Load — Bike-Sharing Dataset

```
bike_data <- read.csv("seoul_bike_sharing.csv")
head(bike_data)    # Inspect first rows
str(bike_data)     # Check data types and column names
```

4. Standardise, Save & Summarise Weather Data

```
colnames(weather_data) <- tolower(colnames(weather_data))
write.csv(weather_data, "weather_data.csv", row.names = FALSE)
file.exists("weather_data.csv")    # Confirm export
summary(weather_data)              # Descriptive statistics
```

Source: 01_data_collection_R.ipynb

A2 — Data Wrangling Code

1. Load, Inspect & Remove NAs

```
bike_data <- read.csv("seoul_bike_sharing.csv")
head(bike_data)
str(bike_data)

# Check for missing values
colSums(is.na(bike_data))

# Remove rows with any NA
bike_data <- na.omit(bike_data)
```

2. Column Names & Type Casting

```
# Standardise column names
colnames(bike_data) <- tolower(
  gsub(" ", "_", colnames(bike_data)))

# Parse date column
bike_data$date <- as.Date(
  bike_data$date, format = "%d/%m/%Y")

# Cast numeric columns
bike_data$rented_bike_count <-
  as.numeric(bike_data$rented_bike_count)
bike_data$temperature <-
  as.numeric(bike_data$temperature)
bike_data$humidity <-
  as.numeric(bike_data$humidity)
bike_data$wind_speed <-
  as.numeric(bike_data$wind_speed)
```

3. Date Feature Engineering & Export

```
bike_data$year        <- year(bike_data$date)
bike_data$month       <- month(bike_data$date)
bike_data$day_of_week <- weekdays(bike_data$date)

write.csv(bike_data, "clean_bike_data.csv", row.names = FALSE)
file.exists("clean_bike_data.csv")
```

Source: 02_data_wrangling_R.ipynb

A3a — SQL: Setup & Queries 1–4

Setup — in-memory SQLite connection & table registration

```
conn <- dbConnect(SQLite(), ":memory:")
dbWriteTable(conn, "SEOUL_BIKE_SHARING",    seoul_bike_sharing, overwrite=TRUE)
dbWriteTable(conn, "WORLD_CITIES",          world_cities,       overwrite=TRUE)
dbWriteTable(conn, "BIKE_SHARING_SYSTEMS", bike_sharing_systems, overwrite=TRUE)
dbListTables(conn)
```

Q1 — Total row count

```
dbGetQuery(conn, "
  SELECT COUNT(*) AS RECORD_COUNT
  FROM SEOUL_BIKE_SHARING;
")
```

Q2 — Operational hours

```
dbGetQuery(conn, "
  SELECT COUNT(*) AS OPERATIONAL_HOURS
  FROM SEOUL_BIKE_SHARING
  WHERE RENTED_BIKE_COUNT > 0;
")
```

Q3 — Date range

```
dbGetQuery(conn, "
  SELECT MIN(DATE) AS FIRST_DATE,
         MAX(DATE) AS LAST_DATE
  FROM SEOUL_BIKE_SHARING;
")
```

Q4 — Distinct seasons

```
dbGetQuery(conn, "
  SELECT DISTINCT SEASONS
  FROM SEOUL_BIKE_SHARING;
")
```

Source: 03_eda_py.ipynb (SQL section)

A3b — SQL: Queries 5–8

Q5 — Holiday count

```
dbGetQuery(conn, "
  SELECT COUNT(DISTINCT DATE) AS HOLIDAY_COUNT
  FROM SEOUL_BIKE_SHARING
  WHERE HOLIDAY = 'Holiday';
")
```

Q6 — Functioning day breakdown

```
dbGetQuery(conn, "
  SELECT FUNCTIONING_DAY,
         COUNT(*) AS RECORD_COUNT
  FROM SEOUL_BIKE_SHARING
  GROUP BY FUNCTIONING_DAY;
")
```

Q7 — Top-10 busiest (date, hour) pairs

```
dbGetQuery(conn, "
  SELECT DATE, HOUR, RENTED_BIKE_COUNT
  FROM SEOUL_BIKE_SHARING
  ORDER BY RENTED_BIKE_COUNT DESC
  LIMIT 10;
")
```

Q8 — Average demand by season & hour (top 10)

```
dbGetQuery(conn, "
  SELECT SEASONS, HOUR,
         AVG(TEMPERATURE)        AS AVG_TEMPERATURE,
         AVG(RENTED_BIKE_COUNT) AS AVG_HOURLY_DEMAND
  FROM SEOUL_BIKE_SHARING
  GROUP BY SEASONS, HOUR
  ORDER BY AVG_HOURLY_DEMAND DESC
  LIMIT 10;
")
```

Source: 03_eda_py.ipynb (SQL section)

A3c — SQL: Queries 9–10

Q9 — Seasonal demand statistics (avg / min / max / approx std dev)

```
dbGetQuery(conn, "
  SELECT SEASONS,
         AVG(RENTED_BIKE_COUNT) AS AVG_HOURLY_RENT,
         MIN(RENTED_BIKE_COUNT) AS MIN_HOURLY_RENT,
         MAX(RENTED_BIKE_COUNT) AS MAX_HOURLY_RENT,
         SQRT(AVG(RENTED_BIKE_COUNT * RENTED_BIKE_COUNT)
              - AVG(RENTED_BIKE_COUNT) * AVG(RENTED_BIKE_COUNT))
           AS STD_HOURLY_RENT
  FROM SEOUL_BIKE_SHARING
  GROUP BY SEASONS
  ORDER BY AVG_HOURLY_RENT DESC;
")
```

Q10 — Seasonal averages for all weather predictors

```
dbGetQuery(conn, "
  SELECT SEASONS,
         AVG(TEMPERATURE)           AS AVG_TEMPERATURE,
         AVG(HUMIDITY)              AS AVG_HUMIDITY,
         AVG(WIND_SPEED)            AS AVG_WIND_SPEED,
         AVG(VISIBILITY)            AS AVG_VISIBILITY,
         AVG(DEW_POINT_TEMPERATURE) AS AVG_DEW_POINT,
         AVG(SOLAR_RADIATION)       AS AVG_SOLAR_RADIATION,
         AVG(RAINFALL)              AS AVG_RAINFALL,
         AVG(SNOWFALL)              AS AVG_SNOWFALL,
         AVG(RENTED_BIKE_COUNT)     AS AVG_HOURLY_DEMAND
  FROM SEOUL_BIKE_SHARING
  GROUP BY SEASONS;
")
```

Source: 03_eda_py.ipynb (SQL section)

A3d — SQL: Queries 11a–11b & Disconnect

Q11a — Seoul city JOIN

```
dbGetQuery(conn, "
  SELECT C.CITY, C.COUNTRY,
         C.LAT, C.LNG,
         C.POPULATION, B.BICYCLES
  FROM WORLD_CITIES C
  JOIN BIKE_SHARING_SYSTEMS B
    ON LOWER(TRIM(C.CITY))
     = LOWER(TRIM(B.CITY))
   AND LOWER(C.COUNTRY)
         LIKE '%korea%'
   AND LOWER(B.COUNTRY)
         LIKE '%korea%'
  WHERE LOWER(TRIM(C.CITY))
      = 'seoul';
")
```

Q11b — Peer-city fleet JOIN (15,000–20,000 bikes)

```
dbGetQuery(conn, "
  SELECT C.CITY, C.COUNTRY,
         C.LAT, C.LNG,
         C.POPULATION, B.BICYCLES
  FROM WORLD_CITIES C
  JOIN BIKE_SHARING_SYSTEMS B
    ON LOWER(TRIM(C.CITY))
     = LOWER(TRIM(B.CITY))
   AND LOWER(TRIM(C.COUNTRY))
     = LOWER(TRIM(B.COUNTRY))
  WHERE B.BICYCLES
        BETWEEN 15000 AND 20000
  ORDER BY B.BICYCLES DESC;
")
```

Teardown — release SQLite connection

```
# Release in-memory connection; database is destroyed on exit
dbDisconnect(conn)
```

Source: 03_eda_py.ipynb (SQL section)

A4a — ggplot2: Setup, Filter & Charts 1–2

Setup — type casting, filtering, summary

```
seoul_bike <- seoul_bike_sharing %>%
  mutate(
    DATE            = dmy(DATE),
    SEASONS         = factor(SEASONS, levels=c("Spring","Summer","Autumn","Winter")),
    HOLIDAY         = factor(HOLIDAY, levels=c("No Holiday","Holiday")),
    FUNCTIONING_DAY = factor(FUNCTIONING_DAY, levels=c("No","Yes")),
    HOUR            = factor(HOUR, levels=0:23, ordered=TRUE)
  )
seoul_bike <- seoul_bike %>% filter(FUNCTIONING_DAY == "Yes")
summary(seoul_bike)
```

Chart 1 — Bike rental vs. Date

```
ggplot(seoul_bike,
  aes(x=DATE, y=RENTED_BIKE_COUNT)) +
  geom_point(alpha=0.3, size=0.6,
             color="steelblue") +
  labs(
    title = "Bike rental vs. Date",
    x = "Date",
    y = "Rented Bike Count"
  ) +
  theme_minimal()
```

Chart 2 — Rental vs. Datetime (hour-coloured)

```
ggplot(seoul_bike, aes(
  x=DATE, y=RENTED_BIKE_COUNT,
  color=as.integer(as.character(HOUR)))) +
  geom_point(alpha=0.5, size=0.7) +
  scale_color_viridis_c(
    name="Hour of day", option="C") +
  labs(
    title = "Bike rental vs. Datetime",
    x = "Date",
    y = "Rented Bike Count"
  ) +
  theme_minimal()
```

Source: 03_exploratory_data_analysis_R.ipynb

A4b2 — ggplot2: Chart 4 — Daily Precipitation

Chart 4 — Daily total rainfall & snowfall (aggregation + grouped bars)

```
daily_precip <- seoul_bike %>%
  group_by(DATE) %>%
  summarise(
    TOTAL_RAINFALL = sum(RAINFALL, na.rm=TRUE),
    TOTAL_SNOWFALL = sum(SNOWFALL, na.rm=TRUE),
    .groups = "drop") %>%
  pivot_longer(
    cols = c(TOTAL_RAINFALL, TOTAL_SNOWFALL),
    names_to  = "PRECIP_TYPE",
    values_to = "DAILY_TOTAL") %>%
  mutate(PRECIP_TYPE = recode(PRECIP_TYPE,
    "TOTAL_RAINFALL" = "Rainfall (mm)",
    "TOTAL_SNOWFALL" = "Snowfall (cm)"))

ggplot(daily_precip,
  aes(x = DATE,
      y = DAILY_TOTAL,
      fill = PRECIP_TYPE)) +
  geom_col(position="identity",
           alpha=0.7, width=1) +
  scale_fill_manual(values = c(
    "Rainfall (mm)" = "steelblue",
    "Snowfall (cm)" = "lightblue")) +
  labs(
    title = "Daily total rainfall and snowfall",
    subtitle = "Daily totals from hourly observations",
    x = "Date",
    y = "Daily Total",
    fill = "Precipitation type") +
  theme_minimal()
```

Source: 03_exploratory_data_analysis_R.ipynb

A4b — ggplot2: Charts 3 & 5

Chart 3 — Histogram + KDE overlay

```
ggplot(seoul_bike,
  aes(x=RENTED_BIKE_COUNT)) +
  geom_histogram(
    aes(y=..density..),
    bins=30,
    fill="steelblue",
    color="white",
    alpha=0.8) +
  geom_density(
    color="red",
    size=0.9) +
  labs(
    title="Bike rental histogram",
    x="Rented Bike Count",
    y="Density") +
  theme_minimal()
```

Chart 5 — Boxplot by hour of day

```
ggplot(seoul_bike,
  aes(x=HOUR,
      y=RENTED_BIKE_COUNT)) +
  geom_boxplot(
    fill="steelblue",
    alpha=0.6,
    outlier.color="grey60",
    outlier.size=0.5) +
  labs(
    title="Rented bike count by hour",
    x="Hour of day",
    y="Rented Bike Count") +
  theme_minimal()
```

Source: 03_exploratory_data_analysis_R.ipynb

A4d — ggplot2: Chart 7 — Pearson Correlation Heatmap

Chart 7 — Pearson correlation matrix heatmap

```
numeric_predictors <- seoul_bike %>%
  select(
    RENTED_BIKE_COUNT,
    TEMPERATURE,
    HUMIDITY,
    WIND_SPEED,
    VISIBILITY,
    DEW_POINT_TEMPERATURE,
    SOLAR_RADIATION,
    RAINFALL,
    SNOWFALL)

cor_matrix <- cor(numeric_predictors,
                  use = "complete.obs")
round(cor_matrix, 3)

cor_long <- as.data.frame(cor_matrix) %>%
  tibble::rownames_to_column("VAR1") %>%
  pivot_longer(-VAR1,
    names_to  = "VAR2",
    values_to = "CORRELATION")

ggplot(cor_long,
  aes(x = VAR1, y = VAR2,
      fill = CORRELATION)) +
  geom_tile(color = "white") +
  geom_text(aes(
    label = sprintf("%.2f", CORRELATION)),
    size = 3) +
  scale_fill_gradient2(
    low = "red", mid = "white",
    high = "steelblue",
    midpoint = 0,
    limits = c(-1, 1)) +
  labs(
    title = "Pearson correlation among numeric predictors",
    x = NULL, y = NULL,
    fill = "Correlation") +
  theme_minimal() +
  theme(axis.text.x = element_text(
    angle = 45, hjust = 1))
```

Source: 03_exploratory_data_analysis_R.ipynb

A4c — ggplot2: Chart 6 — Temperature vs. Demand by Season

Chart 6 — Temperature vs. demand by season (LOESS facets)

```
ggplot(seoul_bike,
  aes(x = TEMPERATURE,
      y = RENTED_BIKE_COUNT)) +
  geom_point(
    alpha = 0.3,
    size  = 0.6,
    color = "darkgreen") +
  geom_smooth(
    method = "loess",
    se     = FALSE,
    color  = "red",
    size   = 0.7) +
  facet_wrap(~ SEASONS, ncol = 2) +
  labs(
    title    = "Temperature vs. rented bike count, by season",
    subtitle = "LOESS fit per season highlights non-linear slopes",
    x = "Temperature (°C)",
    y = "Rented Bike Count") +
  theme_minimal()
```

Source: 03_exploratory_data_analysis_R.ipynb

A4e | EDA Key Findings Summary

Source: 03_exploratory_data_analysis_R.ipynb

Temperature drives demand

- Strong positive correlation with bike rentals
- Demand peaks at 25–30°C and drops sharply below 5°C
- Dominant predictor across all model types

Humidity suppresses rentals

- Negative relationship: high humidity deters cycling
- Effect amplified in summer months
- Captured via humidity + humidity^2 polynomial terms

Precipitation has sharp impact

- Even low rainfall (<1 mm) reduces demand significantly
- Snowfall shows weaker effect than rain
- Faceted bar chart confirms city-level consistency

Strong hourly seasonality

- Bimodal daily pattern: peaks at ~08:00 and ~18:00
- Weekend profile flatter — leisure not commute driven
- Hour is a high-leverage numeric predictor (coef +27.95)

Season shapes baseline demand

- Autumn highest, Winter lowest across all cities
- Spring–Summer transition shows steepest growth
- Season dummies capture non-linear calendar effects

Non-linear thermal response

- LOESS smooth reveals inverted-U shape vs temperature
- Polynomial term (temp^2) needed to capture the curve
- Heatmap confirms temp × hour interaction at peak times

A5 | Lasso Model Coefficients (λ via cv.glmnet)

λ.min selected by 10-fold cross-validation · glmnet · Source: 08_final_model_selection_R.ipynb

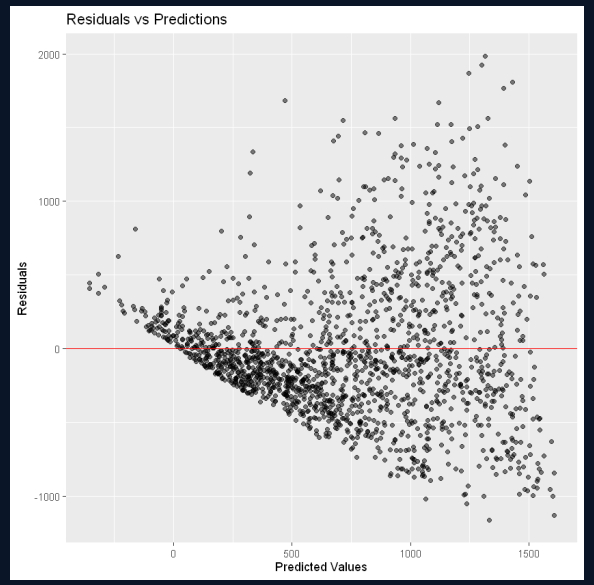
Predictor	Coefficient
(Intercept)	-74.34
temperature	+44.47
humidity	+14.27
wind_speed	+26.50
visibility	+0.002
solar_radiation	-94.62
rainfall	-45.59
snowfall	+7.48
temperature_sq	-0.111
humidity_sq	-0.165
temp_humidity	-0.295

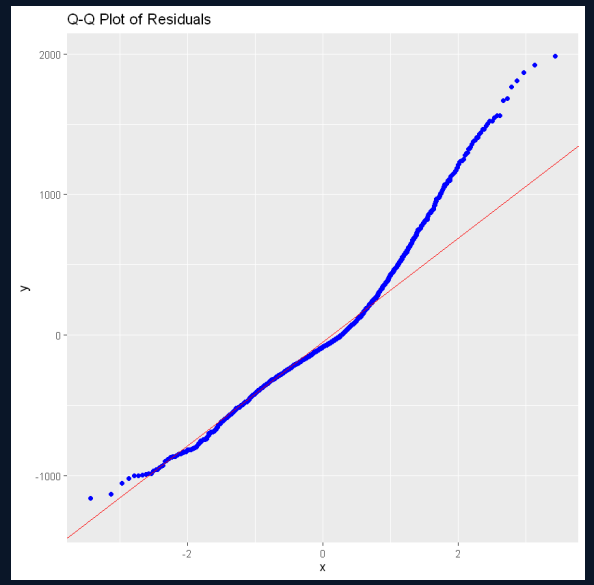
Predictor	Coefficient
seasonsSummer	+38.49
seasonsAutumn	+132.78
seasonsWinter	-191.54
holidayHoliday	-116.78
day_of_weekTuesday	+27.80
day_of_weekWednesday	+65.96
day_of_weekThursday	+27.58
day_of_weekFriday	+45.64
day_of_weekSaturday	-30.59
day_of_weekSunday	-96.02
hour	+27.95

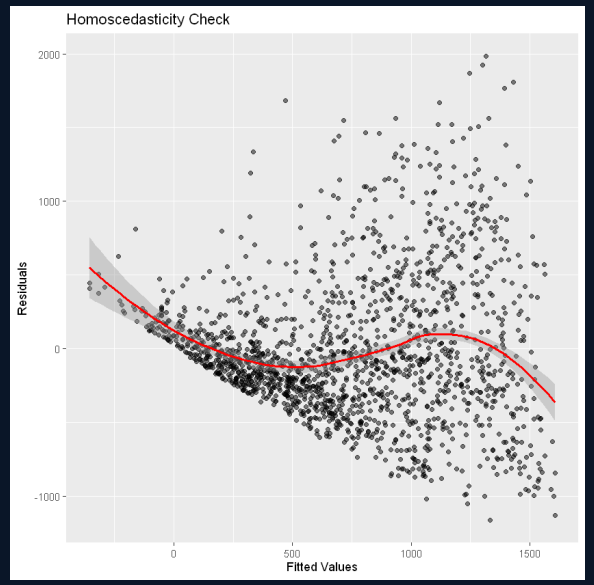
Note: Positive coefficients (green) increase predicted demand; negative (red) decrease it. Reference predictor levels: Spring, No Holiday, Monday.

A6 | Regression Diagnostics (Lasso Experiment)

Source: 06_model_evaluation_diagnostics_R.ipynb







Predictor	VIF	Interpretation
temperature	12.03	High (collinear with temp^2)
humidity	29.83	Very high (collinear with hum^2)
wind_speed	1.15	Acceptable
temperature_sq	4.91	Moderate (expected for polynomial)
humidity_sq	30.56	Very high (expected for polynomial)
temp_humidity	13.07	High (interaction term)

- Residuals vs Fitted: fan-shaped spread indicates heteroscedasticity — variance grows with predicted demand
- Q-Q plot: residuals follow the normal reference line in the centre but deviate at both tails — slight heavy-tailed distribution
- Homoscedasticity check: LOESS trend not flat — confirms non-constant variance; assumptions partially violated
- VIF: high multicollinearity expected for polynomial/interaction terms; Lasso regularisation mitigates coefficient instability

A7 | Shiny App — Source Code Overview

ui.R · server.R — key panels and reactive handlers

ui.R — Right Sidebar Layout

```
tags$div(class = "dash-right",
  conditionalPanel(
    condition =
      "input.city_dropdown == 'All'",
    # City comparison bar chart
    tags$div(class = "chart-card",
      tags$div(class = "chart-header",
        tags$span(
          "Peak Demand by City - Next 24h"),
        actionButton("expand_compare", "",
          class = "btn-expand",
          icon("fullscreen",
            lib = "glyphicon"),
          title = "Expand chart")
      ),
      plotOutput("city_compare_chart",
                 height = "220px")
    )
  ),
  leafletOutput("city_bike_map",
    height = "calc(100vh - 130px)")
)
```

server.R — Click-to-Inspect Handlers

```
# Click-to-inspect handlers — one per chart
# renderText() reads input$*_click
# — Bike demand chart click —
output$bike_click_output <-
  renderText({
  click <- input$bike_click
  clicked_time <- as.POSIXct(
    click$x, origin="1970-01-01", tz="UTC")
  paste0("Time=", format(clicked_time,
    "%H:%M UTC"), "\nPredicted Bikes=",
    scales::comma(round(click$y)))
})
# — Temperature chart click —
output$temp_click_output <-
  renderText({
  click <- input$temp_click
  clicked_time <- as.POSIXct(
    click$x, origin="1970-01-01", tz="UTC")
  paste0("Time=", format(clicked_time,
    "%H:%M UTC"), "\nTemperature=",
    round(click$y, 1), " °C")
})
```

Source: ui.R (lines 349–378), server.R (lines 343–372) — Seoul Bike Share Shiny App

A8 | References

Data sources, research papers, and course materials

Academic & Dataset Sources

[1] Sathishkumar V. E., Park J., and Cho Y. (2020). *Using data mining techniques for bike sharing demand prediction in metropolitan city.* Computer Communications, 153, 353–366. https://doi.org/10.1016/j.comcom.2020.02.007

[2] Seoul Bike Sharing Demand Dataset. UCI Machine Learning Repository. https://archive.ics.uci.edu/ml/datasets/Seoul+Bike+Sharing+Demand

Course & Lab Materials

[3] Luo Y., Grossman J., and Ahuja R. *et al.* (Instructors). *Applied Data Science Capstone with R.* IBM Skills Network / Coursera. https://www.coursera.org/learn/applied-data-science-capstone-with-r

[4] OpenWeather. (2024). *One Call API 3.0 — 5-day forecast.* https://openweathermap.org/api/one-call-3